

Custom Kernel Support Vector Classifier

1. Initialization Parameters

- $h_{bounds} = (h_{min}, h_{max})$: Search interval for kernel hyperparameter h .
- `kernel_fit_type`: String specifying method for initial kernel parameter estimation.
- `objective_metric`: String ('spatial' or 'axis') selecting the objective function for h optimization.
- `svm_kwargs`: Arguments for the underlying `sklearn.svm.SVC`.

2. Hyperparameter Optimization Objective Criterion

This criterion evaluates a candidate kernel hyperparameter $h_{candidate}$.

Inputs:

- $h_{candidate}$: Candidate value for h .
- $\mathbf{X} \in \mathbb{R}^{N \times D}$: Training data matrix (N samples, D features).
- $\mathbf{y} \in \mathcal{Y}^N$: Training class labels. Let $\mathcal{C} = \{C_0, C_1, \dots, C_{K-1}\}$ be K unique classes.

Pre-requisites (from fit method):

- $\eta \in \mathbb{R}$, Σ^{-1} : Initial parameters from `kernel_fit(X)`.

Calculation Steps:

1. **Parameterized Kernel Matrix Computation:** Compute $\mathbf{K}_h \in \mathbb{R}^{N \times N}$ where $(\mathbf{K}_h)_{uv} = \text{kernel}(\mathbf{x}_u, \mathbf{x}_v; \eta, \Sigma^{-1}, h_{candidate})$. The function `kernel(·)` is an external procedure.
2. **Data Projection Vector Computation:** For $i = 1, \dots, N$:

$$\begin{aligned} (Q_{C_0}(h))_i &= \sum_{j \text{ s.t. } y_j = C_0} (\mathbf{K}_h)_{i,j} \\ (Q_{C_1}(h))_i &= \sum_{j \text{ s.t. } y_j = C_1} (\mathbf{K}_h)_{i,j} \end{aligned}$$

Yielding $Q_{C_0}(h), Q_{C_1}(h) \in \mathbb{R}^N$. $C_0 = \mathcal{C}[0], C_1 = \mathcal{C}[1]$.

3. **Objective Score Calculation ($S(h)$):**

- If 'spatial': $S(h) = S_{spatial}(Q_{C_0}(h), Q_{C_1}(h), \mathbf{y})$.

- If 'axis': $S(h) = S_{axis}(Q_{C_0}(h), Q_{C_1}(h), \mathbf{y})$.

$S_{spatial}$ and S_{axis} are externally defined objective functions.

4. **Return Value:** $-S(h)$. If $S(h)$ is NaN or $K < 2$, ∞ is returned.

3. Model Training Procedure (fit)

Inputs: $\mathbf{X} \in \mathbb{R}^{N \times D}$ (training data), $\mathbf{y} \in \mathcal{Y}^N$ (training labels).

Procedure:

1. Validate inputs. Store $\mathbf{X}_{train} = \mathbf{X}$. Identify unique classes $\mathcal{C}$.
2. $(\eta, \Sigma^{-1}) = \text{kernel_fit}(\mathbf{X}_{train}, \text{type} = \text{kernel_fit_type})$.
3. Optimize h :

$$h_{opt} = \arg \max_{h \in [h_{min}, h_{max}]} S(h)$$

This is achieved by minimizing $-S(h)$ using a scalar optimization routine.

4. Compute final training kernel: $\mathbf{K}_{train_opt} = \text{kernel}(\mathbf{X}_{train}, \mathbf{X}_{train}, \eta, \Sigma^{-1}, h_{opt})$.
5. Train SVM: Initialize $\text{SVC}(\text{kernel}='precomputed', \dots)$. Fit with $\text{SVC.fit}(\mathbf{K}_{train_opt}, \mathbf{y})$.

4. Prediction Procedure (predict)

Inputs: $\mathbf{X}_{new} \in \mathbb{R}^{M \times D}$ (new samples).

Procedure:

1. Validate $\mathbf{X}_{new}$.
2. Compute test kernel matrix $\mathbf{K}_{test} \in \mathbb{R}^{M \times N}$:

$$\mathbf{K}_{test} = \text{kernel}(\mathbf{X}_{new}, \mathbf{X}_{train}, \eta, \Sigma^{-1}, h_{opt})$$

3. Predict: $\hat{\mathbf{y}}_{new} = \text{SVC.predict}(\mathbf{K}_{test})$.

Output: $\hat{\mathbf{y}}_{new} \in \mathcal{Y}^M$ (predicted labels).